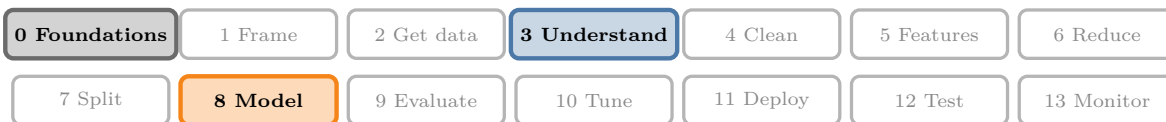


Naive Bayes with Numerical Data: Gaussian Naive Bayes

Contents

1. Overview	2
2. The problem	2
3. The assumption: normal distributions	3
4. The numbers	4
5. When the data is not normal	6
6. Summary	7
7. Sources	7
8. Key terms	7

Where this fits: Pipeline map steps 0 (Foundations), 3 (Understand data), 8 (Model). See the [Course map](#).



- **Builds on:** Classification problems ([Note 3](#)); Independent and mutually exclusive events ([Note 84](#)); Bayes' theorem ([Note 86](#)).
- **Leads to:** Cumulative distribution function (CDF) ([Note 241](#)); Density estimation ([Note 243](#)); Z-score outlier method ([Note 251](#)); Standard normal and the z-table ([Note 251](#)); Q-Q plot ([Note 260](#)); Central limit theorem ([Note 271](#)).
- **Compare with:** Student's t-distribution ([Note 282](#)); Likelihood ([Note 630](#)).

1. Overview

Key point: Numerical features rarely repeat exactly, so counting fails. Gaussian Naive Bayes assumes each feature follows a normal distribution within each class, and uses the normal curve's height (density) in place of a counted probability.

So far every **feature** (G-772; input variable, one column of the data table) was categorical, so probabilities came from counting. This Note handles **numerical** features such as height or weight, and introduces **Gaussian Naive Bayes** (G-830), the most common variant for them.

2. The problem

Key point: No man in the data is exactly 185 cm tall, so $P(\text{height} = 185 \mid \text{male})$ by counting is 0.

The data has 8 people. Each person is one **observation** (G-1374; one record, a row of the table). The two features are height (cm) and weight (pounds); the **target** (G-1949; the output we predict) is gender:

Height (cm)	Weight (lb)	Gender
182.9	180	male
180.4	190	male
170.1	170	male
180.4	165	male
152.4	100	female
167.6	150	female
165.2	130	female
175.3	150	female

A new person is 185 cm tall and weighs 170 pounds. Male or female?

The Naive Bayes recipe (the maths Note) needs

$$P(\text{male}) \times P(\text{height} = 185 \mid \text{male}) \times P(\text{weight} = 170 \mid \text{male})$$

and the same for female. The **prior** (G-1565), the share of each class in the data, is easy: 4 of 8 are male, so $P(\text{male}) = 1/2$. But no man in the data is exactly 185 cm, so counting gives $P(\text{height} = 185 \mid \text{male}) = 0$, and the whole score is 0. With a continuous measurement, almost every new value is one never seen before, so counting cannot work.

$P(\text{height} = x \mid \text{male})$ by counting the 4 men

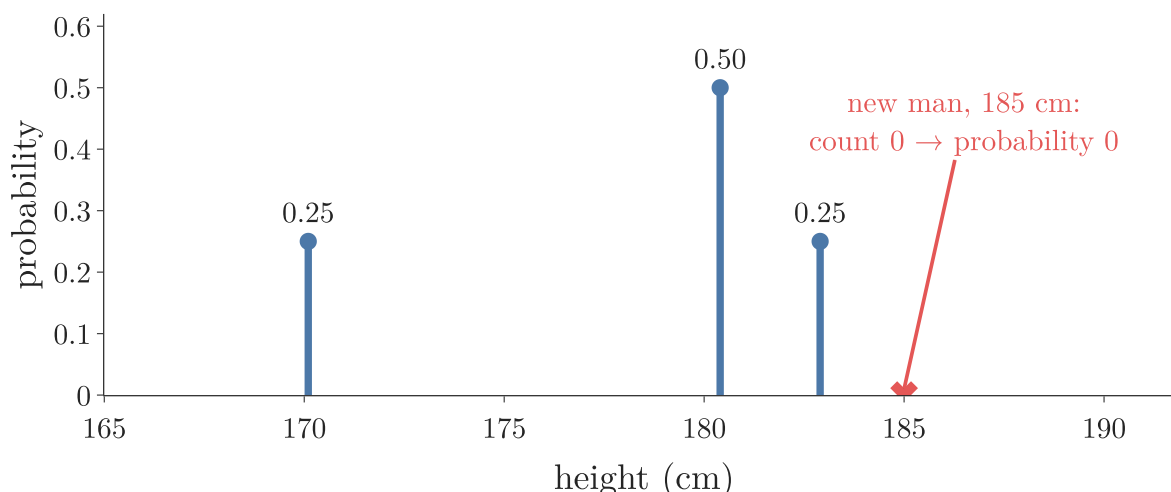


Figure 1: $P(\text{height} \mid \text{male})$ by counting: a spike at each of the 4 male heights, and nothing at 185 cm

In Figure 1, watch the gap at 185 cm: counting puts all its probability on the exact heights already seen, so a man just 2 cm taller than the tallest one gets probability 0.

3. The assumption: normal distributions

Key point: Assume each feature is normally distributed within each class. Estimate the mean and standard deviation, then read off the curve's height at the new value.

Gaussian Naive Bayes assumes that, within each class, each numerical feature follows a **normal** (Gaussian) **distribution** (G-1343), the bell curve of the earlier statistics Notes.

For each class and each feature:

1. compute the **mean** (G-1203) μ and **standard deviation** (G-1871) σ of that feature over that class's observations;
2. for a new value x , compute the height of that class's normal curve at x , the **probability density** (G-1569):

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

3. use $f(x)$ in place of $P(x \mid \text{class})$ in the Naive Bayes product.

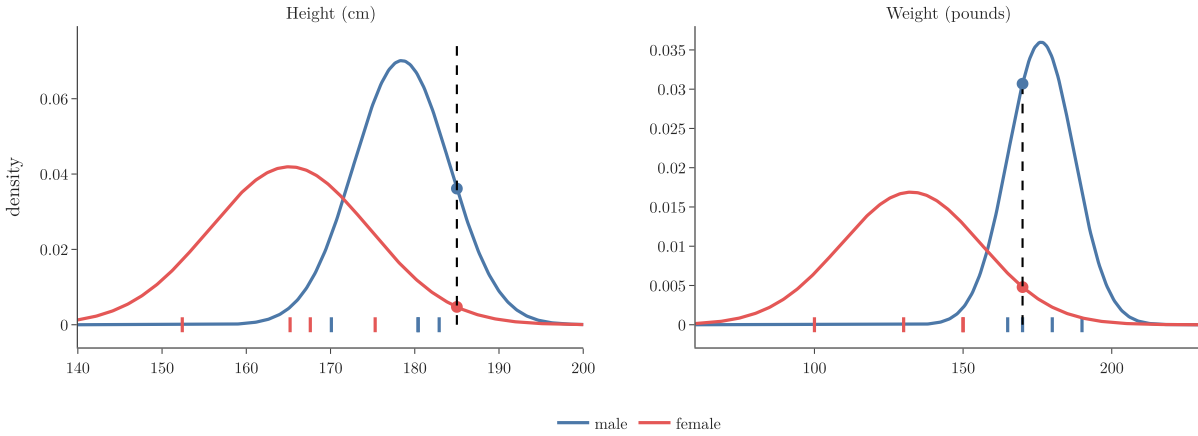


Figure 2: A normal curve per class for each feature; the dots mark the new person's values

Figure 2 shows the four fitted curves. The new person's height (185 cm) sits on the right flank of the male curve but far out in the tail of the female curve; the same holds for the weight (170 pounds).

4. The numbers

Key point: Male score 5.5×10^{-4} against female 1.1×10^{-5} : predict male, with about 98% probability.

Class	Feature	Mean	Std. deviation	Density at the new value
male	height	178.45	5.69	0.03615
male	weight	176.25	11.09	0.03070
female	height	165.12	9.51	0.00473
female	weight	132.50	23.63	0.00479

$$\text{male: } 0.5 \times 0.03615 \times 0.03070 = 5.5 \times 10^{-4}$$

$$\text{female: } 0.5 \times 0.00473 \times 0.00479 = 1.1 \times 10^{-5}$$

The male score is about 49 times larger, so the prediction is **male**. As probabilities: 98% male, 2% female.

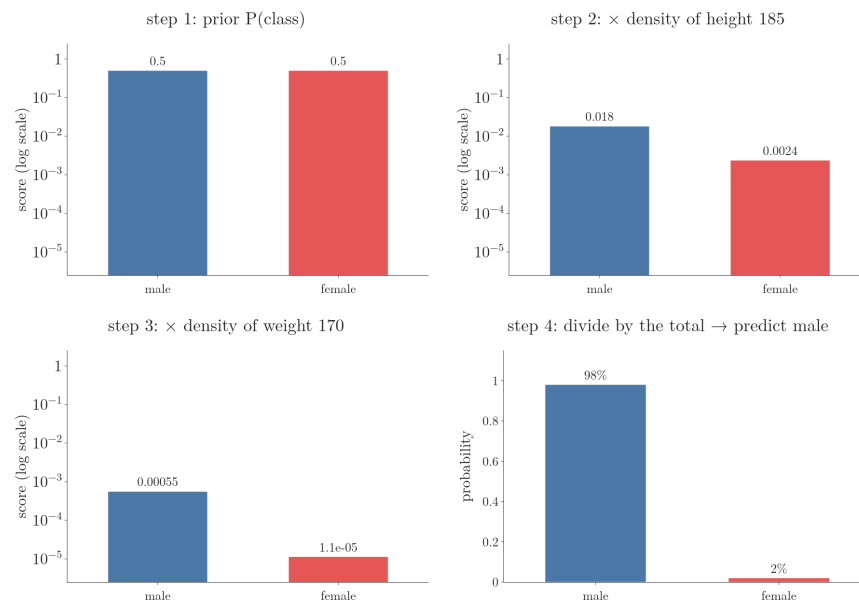


Figure 3: The two scores built one factor at a time (log scale), then divided by their total

In Figure 3, watch the gap open: both classes start at 0.5, the height density pulls the female score about 8 times lower, and the weight density widens the gap to about 49 times.

Extra: A density is not a probability: for a continuous variable, the probability of exactly 185.000... cm is 0, and densities can even be larger than 1. But the density measures how likely values **near** 185 are, and the same small interval would multiply every class's density equally. So comparing densities across classes gives the same decision as comparing probabilities.

Python: scikit-learn's GaussianNB.

```
from sklearn.naive_bayes import GaussianNB
```

```
nb = GaussianNB().fit(df[["height_cm", "weight_lb"]], df["gender"])
nb.theta_           # class means
nb.var_             # class variances
nb.predict_proba(new_person) # about 99% male
```

GaussianNB estimates the variance by dividing by n rather than $n - 1$ (and adds a tiny amount, `var_smoothing`, for numerical safety; scikit-learn docs, `GaussianNB`), so its numbers differ slightly from the table: 99.2% male instead of 98.0%. The prediction is the same.

5. When the data is not normal

Key point: The normal assumption can be wrong. Other variants assume other distributions: multinomial for counts, Bernoulli for yes/no, categorical for categories.

The normal assumption is a choice, and it can be poor, for example for a skewed feature like income. Naive Bayes can use any distribution that suits the data, giving different variants:

Variant	Assumed distribution	Typical features	scikit-learn
Gaussian	normal	continuous measurements	<code>GaussianNB</code>
Multinomial	multinomial	counts, such as word counts in a document	<code>MultinomialNB</code>
Bernoulli	Bernoulli (yes/no)	binary features, such as “word present or not”	<code>BernoulliNB</code>
Categorical	categorical	categories, as in the Play Tennis Note	<code>CategoricalNB</code>

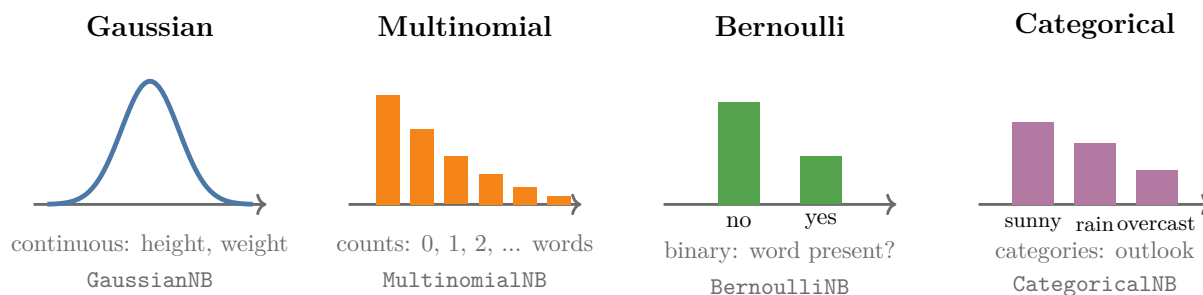


Figure 4: The shape each variant assumes for one feature within one class

In Figure 4, match the shape to the feature: a smooth bell for measurements, bars over 0, 1, 2, ... for counts, two bars for yes/no, one bar per category.

Each variant suits one kind of data (scikit-learn user guide §1.9), so we look at each feature’s distribution and pick the variant whose assumption fits it. A strongly skewed numerical feature can also be transformed first (the power transformer Note) so that it looks more normal.

6. Summary

- Numerical values rarely repeat, so $P(x \mid \text{class})$ cannot be counted.
- Gaussian Naive Bayes fits a normal curve (mean, standard deviation) per class and feature, and uses the density at x .
- New person 185 cm, 170 lb: male score 5.5×10^{-4} , female 1.1×10^{-5} : male (98%).
- Other variants (multinomial, Bernoulli, categorical) suit other kinds of features.

7. Sources

Built from

- CampusX, “Naive Bayes Part 9 | Handling Numerical Data”, YouTube, <https://www.youtube.com/watch?v=TCgK2nBJx9o>

Other references

- **scikit-learn docs:** `sklearn.naive_bayes.GaussianNB` (`var_smoothing`); user guide Section 1.9, “Naive Bayes”, scikit-learn 1.9.

8. Key terms

Term	Meaning
Gaussian Naive Bayes	Naive Bayes that models each numerical feature as normally distributed within each class
Probability density	The height of a continuous distribution’s curve; compares how likely nearby values are
GaussianNB	scikit-learn’s Gaussian Naive Bayes
MultinomialNB	Naive Bayes for count data, such as word counts
BernoulliNB	Naive Bayes for binary (yes/no) features